

Adaptive Room Planner — PRD

DivHacks 2026 · Product functionality and core user flow

Sep 26, 2026 · @Siddhi Mhatre · DivHacks 2026 · Draft v0.1

1. Summary

Adaptive Room Planner is a spatial room-planning assistant that scans a real room, creates a persistent 3D digital twin, and lets users test furniture or lifestyle changes before physically rearranging or buying anything.

The user scans once with RoomPlan, rebuilds their current room from preset furniture in a top-down, Sims-like editor, and then asks for changes in plain language, in the app or over iMessage through Photon. Gemini turns each request into structured spatial constraints, Backboard remembers the user's non-negotiables, and every answer becomes a named layout variant that never overwrites the Current Room.

Demo promise: send a Facebook Marketplace desk listing to the Photon agent, ask "Will this fit beside my window without moving my bed?", and get a saved "Marketplace Desk" variant you can adjust and compare in under two minutes.

2. Problem and opportunity

People in small apartments, dorms and shared rooms cannot see how their space could work before they commit to moving furniture or buying something new.

- Existing planners start from an empty room and ignore what the user already owns and cannot move.
- Marketplace shopping happens on a phone with a tape measure and a screenshot; "will it fit" is guesswork.
- Rearranging is physical labor, so people try one layout and stop, even when a better one exists.
- Preferences like "never move my bed" or "keep the desk near natural light" live in the user's head and have to be repeated every time.

The opportunity is a persistent digital twin that is cheap to capture (one iPhone scan), cheap to query (a text message), and safe to experiment with (variants, not overwrites). Future applications beyond the hackathon include real-estate staging, architecture and

interior-design proposals, and small-business floor planning.

3. Goals, success metrics, and non-goals

The build succeeds if the full loop (scan or load, edit, message, interpret, variant, compare) runs live in front of judges without a fallback.

Goals

- Capture a real room once and keep its empty skeleton as a permanent baseline.
- Let the user rebuild their current room in minutes from preset, real-sized furniture.
- Accept furniture and requests from the real world: a photo, a product link, or a sentence, in the app or over iMessage.
- Turn requests into structured constraints that respect locked items and stated preferences.
- Produce named layout variants that can be adjusted, validated for fit, saved, and compared.

Success metrics for the demo

Metric	Target
Scan or sample room to rendered editor	Under 30 s
Rebuild current room (5 items)	Under 2 min of dragging
Photon message to placed item in a variant	Under 20 s round trip
Fit validation on a change	Under 100 ms, in the browser
Demo run end to end	Under 2 min, no crashes, live network

Non-goals

- ElevenLabs voice, WebXR, and full AR preview.
- Perfect 3D generation from photos; preset assets scaled to real dimensions are enough.
- Advanced styling or aesthetics-driven interior design.
- Complex automatic layout optimization; the MVP places what was asked and validates it, it does not re-solve the whole room.

- Multi-room planning, purchasing, or price tracking beyond reading a listing's dimensions and price.

4. Target users

The primary user is a student or young renter in a roughly 100–150 sq ft room who owns a fixed set of furniture, shops secondhand, and wants one more thing to fit without a weekend of trial and error.

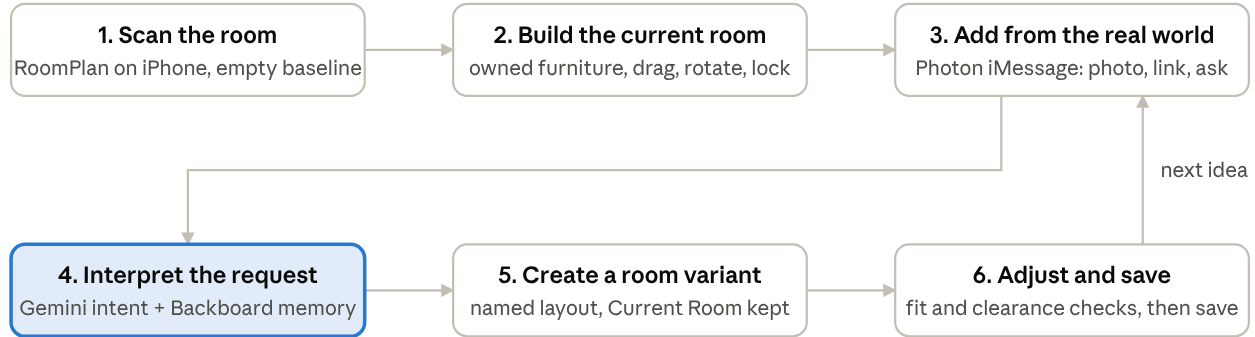
User	Situation	What they ask the room
Marketplace shopper (demo persona)	Found a desk on Facebook Marketplace, bed cannot move	"Will this fit beside my window without moving my bed?"
Lifestyle changer	Wants a yoga or workout zone in a bedroom	"Make space for yoga without moving my bed."
Roommate	Shares a room and needs two desks	"Give us both a desk near an outlet."
Mover	Deciding what to keep before a move (packing puzzle)	"Does my current furniture fit in the new room?"

Secondary users, out of scope for the demo but worth a slide: real-estate agents staging listings, architects and interior designers testing proposals, and small businesses laying out a studio or shop.

5. Core user flow

Six steps; the Current Room is never overwritten, and every request from step 3 onward produces a new named variant that loops back into the editor.

Scan once, then every idea becomes a named variant



core user flow · 6 steps, one loop

1. Scan the room. Scan on iPhone with RoomPlan. Capture room shape, dimensions, walls, doors and windows. Save a clean unfurnished room skeleton as the permanent baseline.
2. Build the current room. Add furniture the user already owns, using preset 3D assets scaled to real dimensions. Drag, rotate and lock items in a top-down, Sims-like 3D editor. Save as "Current Room".
3. Add something from the real world. Send the Photon iMessage agent a furniture photo, a marketplace or product link, or a request such as "I need a desk under \$100 that fits beside my window."
4. Interpret the request. Gemini extracts furniture details (type, dimensions, price) and user intent. Backboard supplies persistent preferences such as "never move my bed" or "keep the desk near natural light."
5. Create a room variant. Generate a named layout such as "Marketplace Desk", "Yoga Layout" or "Reading Corner" without overwriting the Current Room.
6. Adjust and save. The user can move, rotate, remove, add or lock furniture. Geometry checks confirm that objects fit and required clearances remain, then the variant is saved and can be compared with the Current Room.

6. Functional requirements

P0 is the nine key functionalities at MVP depth; P1 deepens them once the demo path runs end to end; P2 is slide material.

Functionality	Priority	Requirement	Acceptance criteria
Room scanning	P0	Capture irregular geometry and dimensions with RoomPlan on iPhone and export as JSON. Fallbacks: load a bundled sample room, or type length, width, height and place doors and windows by hand	A scanned or loaded room appears in the editor with walls, doors and windows in the right places
Empty-room baseline	P0	Store the physical room without furniture as the permanent skeleton every layout builds on	The skeleton is saved once per room; deleting or editing a layout never changes it
3D room editor	P0	Three.js top-down, Sims-like editor: add, drag, rotate, lock and remove furniture from preset assets scaled to real dimensions	Drag on the floor plane, rotate in 90° steps, snap to walls, visible lock badge, undo
Furniture import	P0	Accept a photo, a product or marketplace link, or manual dimensions; produce a furniture item with type, size and price	A listing link becomes a placeable object of the right size within 10 s; a photo yields at least type and an estimated size
AI spatial request	P0	Gemini turns "make space for yoga without moving my bed" into an intent plus structured constraints (locks, clear zones, adjacency, keep-clear)	The request returns a valid plan JSON; unknown intents return a clarifying question
Layout variants	P0	Save multiple named arrangements per room; the Current Room is never overwritten; variants can be renamed, duplicated and deleted	"Marketplace Desk" exists beside "Current Room" and both reopen after a reload

Functionality	Priority	Requirement	Acceptance criteria
Fit validation	P0	Check room bounds, item overlap, door clearance and requested open-space zones on every change (rules in section 8)	Violations are listed by item name; a bad placement is flagged in red, never silently allowed
iMessage access	P0	Photon receives links, images and quick questions; the agent replies with the result and a link into the editor	One live message round trip works in the demo
Compare view	P1	Current Room vs. a variant side by side with open floor %, conflicts and a list of what moved	Both layouts and their numbers on one screen
Preference memory	P1	Backboard stores non-negotiables ("never move my bed") and long-term preferences ("desk near natural light") across sessions	A second session applies a preference the user did not repeat
Space analysis overlays	P1	Open floor %, walkable / low-clearance / keep-clear floor overlays, reachable storage	Overlays and numbers update on every drag
Voice and XR	P2	Web Speech input; view a variant at true scale on a headset	Out of MVP; slide only

7. AI request handling

Gemini never places furniture directly: it produces intent and structured constraints, the backend proposes a placement, and the browser's fit validation has the final say, so results are explainable and never overlap walls.

Inputs to the agent: the request (typed in the app, or a Photon message with optional photo or link), the room skeleton, the Current Room or active variant with locks, and Backboard memory for the user.

Output: one plan JSON that the frontend animates into a new variant.

```
{
  "intent": "fit_item",
  "item": {"type": "desk", "w_m": 1.2, "d_m": 0.6, "h_m": 0.75, "price_usd": 80, "source": "marketplace_link"},
  "constraints": [
    {"type": "lock", "item": "bed"},
    {"type": "adjacent", "item": "desk", "feature": "window"},
    {"type": "clear_zone", "w_m": 1.8, "d_m": 1.2, "label": "Yoga zone"},
    {"optional": true}
  ],
  "placement": {"item": "desk", "x": 0.3, "y": 2.1, "rotation": 90},
  "variantName": "Marketplace Desk",
  "explanation": "Placed the desk on the window wall; the bed stays where it is."
}
```

Intents for the MVP

Intent	Example	Constraint types produced
fit_item	"Will this desk fit beside my window without moving my bed?" (link, photo or dimensions; optional budget)	item, lock, adjacent
make_space	"Make space for yoga without moving my bed"	clear_zone, lock
keep_clear	"I don't want my desk blocking the window"	keep_clear (window, door swing, outlet)
compare	"Which layout is better for studying?"	none; returns a ranked explanation of existing variants
clarify	anything else	a question back to the user, in the editor or over iMessage

Preference memory (Backboard): store stated non-negotiables ("never move my bed"), long-term preferences ("keep the desk near natural light"), room purpose, and the last few requests, keyed by user. Every agent call is prefixed with these so the user does not

repeat themselves; a preference the user states in a request is written back the same turn.

Photon conversation: an inbound iMessage with a link or photo runs furniture import first, then the agent. The reply is one sentence ("It fits beside the window; the bed did not move.") plus a link to the variant. A conflict is reported the same way ("It's 12 cm too wide for the window wall; want me to try the wall by the door?").

8. Fit validation rules

Validation runs on a 10 cm floor grid in the browser on every change, so the editor and the agent share one set of rules.

Rule	Check	On failure
Room bounds	Every footprint is inside the wall polygon	Item turns red; cannot save the variant
Overlap	No two footprints intersect	Both items flagged and named
Locked items	Locked items are never moved by the agent or by validation	Plan rejected before placement
Door clearance	The door swing arc and 0.9 m (about 3 ft) in front of the door stay empty	Conflict counted; keep-clear overlay in red
Window keep-clear (optional)	Nothing taller than the sill in the window band when the user asked	Warning, not a block
Access edges	Beds and desks keep one 0.75 m free long edge; wardrobes and dressers keep 0.75 m in front	Low-clearance overlay in yellow
Requested open zones	A requested clear zone (e.g. 1.8 × 1.2 m for yoga) exists as one contiguous free rectangle	Agent reports the largest zone it could find
Walkable path	A flood fill over free cells connects the door to every bed and desk	"Path blocked to desk"

Derived numbers shown in the analysis panel: open floor % (free cells over room area), largest free rectangle in sq ft with what fits in it, walkability (Good / Tight / Blocked), reachable storage % (storage front edges adjacent to free cells), and conflict count.

9. Technical architecture

One FastAPI service sits between three clients (iOS scanner, web editor, Photon webhook) and three services (Gemini, Backboard, MongoDB Atlas); the browser owns geometry and validation so the editor stays responsive.

Layer	Tech	Responsibility
Scanner	Swift + RoomPlan	Scan walls, doors, windows and dimensions; export the empty-room skeleton as JSON and upload it
Frontend	Three.js + TypeScript (React + Vite + react-three-fiber)	Top-down 3D editor, drag/rotate/lock, fit validation, overlays, variants and compare view, request box
Backend	FastAPI	Rooms, furniture, layouts, agent orchestration, placement proposal, Photon webhook
Database	MongoDB Atlas	Room geometry, furniture metadata, saved variants, users and preferences pointer
AI reasoning	Gemini API	Intent and structured spatial constraints; furniture details from links and photos
Memory	Backboard	Persistent preferences and conversational memory per user
Messaging	Photon	iMessage in (links, images, questions) and out (result plus editor link)
3D assets	Blender / Blender MCP	Optional creation or refinement of preset furniture GLBs

Data model (MongoDB collections)

- `rooms`: `_id`, `userId`, `name`, `dimensions` {`l`, `w`, `h`} in meters, `walls`[] {`x1`, `y1`, `x2`, `y2`}, `doors`[] {`wall`, `offset`, `width`, `swing`}, `windows`[] {`wall`, `offset`, `width`, `sillHeight`}, `source` (`scan` / `manual` / `sample`), `createdAt`.
- `furniture`: `_id`, `userId`, `category`, `name`, `size` {`w`, `d`, `h`}, `asset` (preset GLB key), `source` (`preset` / `link` / `photo` / `manual`), `listingUrl`, `imageUrl`, `price`.
- `layouts`: `_id`, `roomId`, `name` ("Current Room" or a variant name), `isCurrent`, `items`[] {`furnitureId`, `x`, `y`, `rotation`, `locked`}, `zones`[] {`label`, `x`, `y`, `w`, `d`}, `metrics` {`openFloor`, `conflicts`, `walkability`, `reachableStorage`}, `createdBy` (`user` / `agent`), `requestText`, `parentLayoutId`.
- `users`: `_id`, `phone` (Photon identity), `backboardUserId`, `createdAt`.

API surface

Method and path	Purpose
POST /rooms	Create a room from RoomPlan JSON, manual dimensions, or the bundled sample
GET /rooms/{id}	Room skeleton plus its layouts
POST /rooms/{id}/layouts	Save a layout (Current Room or a variant)
PATCH /layouts/{id}	Rename, update items and locks, mark as current
POST /furniture/import	Body: link, image, or dimensions. Returns a furniture item with size and price
POST /rooms/{id}/agent	Body: request text, base layout id, optional furniture id. Returns the plan and a new variant
GET /layouts/{a}/compare/{b}	Metric deltas and the list of items that moved
POST /webhooks/photon	Inbound iMessage; routes to import and agent; replies with text and a link

Frontend structure

- `RoomScene`: orthographic top-down camera, floor grid, walls with door and window cutouts, overlay planes.
- `FurnitureItem`: preset GLB or dimension-true box, drag on the floor plane, 90° rotation, lock badge.
- `useFitValidation`: the section 8 rules and derived numbers, memoized per layout.

- Panels: `LibraryPanel`, `RequestBar`, `AnalysisPanel`, `VariantsPanel`, `CompareView`.
- State: one store holding room, active layout, variants, selection and validation results.

Sequence for the demo request: Photon webhook receives the listing link, `POST /furniture/import` extracts size and price with Gemini, `POST /rooms/{id}/agent` builds the prompt with Backboard memory and returns a plan, the backend saves the "Marketplace Desk" variant, Photon replies with the result and a link, and the editor opens the variant with validation already run.

10. Hackathon MVP scope

In scope, in build order:

- ☐ Scan or load one room and save its empty-room skeleton.
- ☐ Render the room in Three.js.
- ☐ Add, move, rotate and lock a small furniture set (bed, desk, chair, dresser, shelf, nightstand).
- ☐ Send one furniture link, photo or text request through Photon.
- ☐ Use Gemini to convert the request into structured constraints and a placement.
- ☐ Create and save one new layout variant without overwriting the Current Room.
- ☐ Compare Current Room vs. Suggested Layout.

Out of MVP: ElevenLabs, WebXR, full AR preview, perfect 3D generation from photos, advanced styling, and complex automatic layout optimization. Each gets one line on the roadmap slide.

11. Demo scenario

A user scans their bedroom, saves the current layout, finds a desk on Facebook Marketplace, sends the listing to the Photon agent and asks "Will this fit beside my window without moving my bed?" The system checks the saved room, creates a "Marketplace Desk" variant, places the desk in 3D, and lets the user adjust and save the result.

Script, about two minutes

Time	On screen	Line
0:00	Phone: RoomPlan scan (pre-recorded video if no LiDAR device at the table)	"One scan, and the room is a digital twin."
0:20	Editor: Current Room with bed locked, desk, dresser	"This is my room today. My bed is locked; it never moves."
0:40	Phone: iMessage to Photon with the Marketplace link and the question	"I found a desk. Will it fit beside my window without moving my bed?"
1:00	Reply arrives; editor opens "Marketplace Desk" variant	"Gemini read the listing, Backboard remembered the bed rule, and here it is by the window."
1:20	Drag the desk into the door swing; red conflict; drag back	"Fit validation catches what I'd only notice after carrying it up three flights."
1:40	Compare view: Current Room vs. Marketplace Desk	"Two layouts, side by side, nothing overwritten. I can buy with confidence."

12. Milestones, team, risks, open questions

Milestones (dates to fill in once the hackathon schedule is confirmed)

Milestone	Done when
M1 Room loads	Scan or sample room saved as an empty skeleton and rendered in Three.js
M2 Editor	Add, move, rotate and lock a small furniture set; fit validation flags overlaps and door clearance
M3 Photon in	One furniture link, photo or text request arrives through Photon and reaches the backend
M4 Gemini plan	Request becomes structured constraints and a placement; Backboard stores the stated preference
M5 Variant	New named layout saved beside the Current Room
M6 Compare	Current Room vs. Suggested Layout side by side

Milestone	Done when
M7 Polish	Cozy top-down look, demo script rehearsed, pitch slides

Suggested team split: one person on the Three.js editor and fit validation, one on FastAPI, Atlas and the placement proposal, one on Gemini prompts, Backboard and the Photon webhook, and one on RoomPlan, Blender assets and the pitch.

Risks and fallbacks

- RoomPlan needs a LiDAR iPhone or iPad and a real room at the venue: ship the bundled sample room and manual draw, and record the scan beforehand.
- Gemini returns a placement that overlaps or ignores a lock: fit validation rejects it and the backend retries once with the violation in the prompt; the user can always place by hand.
- Photon approval or iMessage delivery is slow: keep an "Upload photo / paste link" button in the web app that hits the same import and agent endpoints.
- Listing pages block scraping: accept a pasted screenshot or manual dimensions as the input.
- Scope creep into pretty 3D: preset assets or boxes only, one hero room at most.

Open questions

- ☐ Feet or meters in the UI? The mocks show feet; RoomPlan exports meters.
- ☐ Does Photon reply with a rendered image of the variant, or only a link into the editor?
- ☐ Which Gemini model, and do we use structured output mode for the plan JSON?
- ☐ One shared demo user keyed by phone number, or real sign-in?
- ☐ Is the onboarding wizard (Capture, Define, Arrange) a separate screen or the editor's empty state?